Haneen Alaa Hassan

Depi Day6 c#

Essay Questions

1. Why can't a struct inherit from another struct or class in C#?

A struct is a value type in C#. C# does not support inheritance between structs or between a struct and a class.

However, a struct can implement one or more interfaces.

All structs implicitly derive from "System.ValueType", which itself derives from "System.Object".

2. How do access modifiers impact the scope and visibility of a class member?

Access modifiers control the accessibility and visibility of class members. The main access modifiers are:

- "private": Accessible only inside the same class.
- "public": Accessible from anywhere.
- "internal": Accessible within the same assembly or project.
- "protected": Accessible inside the same class and its derived classes.
- "protected internal": Accessible within the same assembly or from derived classes.
- "private protected": Accessible inside the same class or derived classes within the same assembly.

Access modifiers help protect data and control how different parts of a program can access class members.

3. Why is encapsulation critical in software design?

Encapsulation is the concept of hiding the internal data of an object and controlling access to that data.

It is important because it:

1. **Protects data from unwanted modification.**
2. **Controls how data is accessed and changed.**
3. **Improves security.**
4. **Makes the code easier to maintain.**
5. **Reduces dependencies between different parts of the program.**

For example, we can make a field "private" and use properties or methods to access it safely.

4. What are constructors in structs?

A constructor is a special method used to initialize a struct when it is created.

A struct can have parameterized constructors that initialize its fields with specific values.

For example:

Point p = new Point(10, 20);

The constructor receives "10" and "20" and uses them to initialize "X" and "Y".

Structs also have default initialization behavior, where their fields receive their default values.

5. How does overriding methods like ToString() improve code readability?

Overriding "ToString()" allows us to provide a meaningful string representation of an object.

Instead of displaying only the type name, we can display useful information about the object's data.

For example:

Point: X = 10, Y = 20

This improves readability and makes debugging, logging, and displaying objects easier to understand.

6. How does memory allocation differ for structs and classes in C#?

Structs are value types, while classes are reference types.

A struct variable directly contains its data. When a struct is assigned to another variable or passed by value, its data is copied.

A class variable contains a reference to an object. When a class object is assigned or passed by value, the reference is copied, so two variables can refer to the same object.

Structs can be stored inline or on the stack depending on their context, while class objects are normally allocated on the managed heap.

The important difference is that a struct is a value type and a class is a reference type. It is not always correct to simply say "structs are stored on the stack and classes are stored on the heap."

7. What is a Copy Constructor?

A copy constructor is a constructor that creates a new object by copying the values from an existing object of the same type.

For example:

```
public Employee(Employee employee)
{
Name = employee.Name;
Salary = employee.Salary;
}
```

It can be used like this:

```
Employee employee2 = new Employee(employee1);
```

The new object receives the values of the existing object.

C# does not automatically provide a copy constructor for classes. We can define one when we need it.

8. What is an Indexer?

An indexer allows an object to be accessed using the square bracket "[]" operator, similar to an array.

For example:

```
students[0]
```

Instead of calling a method such as:

students.GetStudent(0)

An indexer is useful when a class represents a collection of objects or values.

Business Examples